

Ingeniería De Software 1

El principio **F.I.R.S.T.** establece buenas prácticas para escribir casos de prueba efectivos:

- **Fast (Rápido):** Las pruebas deben ejecutarse rápidamente.
- **Independent (Independiente):** Cada prueba debe ser aislada, sin depender de otras.
- **Repeatable (Repetible):** Las pruebas deben producir el mismo resultado sin importar el entorno.
- **Self-Validating (Auto-validante):** Cada prueba debe indicar claramente si pasó o falló.
- **Timely (Oportuna):** Las pruebas deben escribirse a tiempo, preferiblemente antes de implementar el código.

S.O.L.I.D.

- Principio de Responsabilidad Única (SRP - Single Responsibility Principle):

Un objeto o clase debería tener una única responsabilidad. Esto significa que una clase debe tener solo una razón para cambiar, lo que facilita su comprensión, reutilización y mantenimiento.

- Principio de Abierto/Cerrado (OCP - Open/Closed Principle):

Las entidades de software deben estar abiertas para su extensión pero cerradas para su modificación. Esto implica que el código debe ser fácilmente extensible mediante la adición de nuevas funcionalidades, sin necesidad de modificar el código existente.

- Principio de Sustitución de Liskov (LSP - Liskov Substitution Principle):

Los objetos de una clase derivada deben poder sustituir a los objetos de la clase base sin afectar la funcionalidad del programa. Esto asegura que las clases derivadas sean compatibles con las clases base y evita resultados inesperados en el código.

- Principio de Segregación de Interfaces (ISP - Interface Segregation Principle):

Los clientes no deben depender de interfaces que no utilizan. Este principio propone dividir interfaces grandes en interfaces más pequeñas y cohesivas, específicas para cada cliente, evitando la implementación de métodos innecesarios.

- Principio de Inversión de Dependencia (DIP - Dependency Inversion Principle):

Los módulos de alto nivel no deben depender de los módulos de bajo nivel. Ambos deben depender de abstracciones. Este principio establece que las dependencias deben ser abstracciones (interfaces o clases abstractas) en lugar de implementaciones concretas, lo que permite una mayor flexibilidad y facilidad para realizar cambios.

R=R(Relaciones de calidad = resultados de calidad)

Conocernos más para colaborar mejor. La relación entre la calidad del trabajo en grupo y los resultados de calidad en el diseño de software es un tema clave en el desarrollo exitoso de proyectos. Un equipo que colabora de manera efectiva puede aprovechar las fortalezas individuales

de sus miembros. Esto permite que el diseño del software sea más robusto, ya que se integra la experiencia y el conocimiento diverso de todos.

La comunicación es vital. Un equipo que se comunica claramente sobre las metas, los desafíos y las soluciones tiende a cometer menos errores y malentendidos, lo que se traduce en menos retrabajo y en una mayor calidad del producto final.

Marco Cynefin

Cynefin clasifica los problemas y situaciones en cinco dominios principales, cada uno con características y enfoques distintos:

1. Dominios Claro (Obvio o Simple)

- Características: Este dominio se aplica a situaciones en las que la relación entre causa y efecto es clara y bien entendida. Las soluciones son evidentes, y hay una única "mejor práctica" que se debe seguir.
- Enfoque: Aquí se utiliza el sentido común, y las decisiones se toman basándose en reglas establecidas o procedimientos estándar.
- Estrategia: Sentir-Categorizar-Responder. Se observa lo que está sucediendo, se clasifica en una categoría conocida, y se responde de acuerdo a las mejores prácticas.

2. Dominios Complicado

- Características: Las relaciones causa-efecto en este dominio son claras, pero no son evidentes para todos. Puede haber múltiples soluciones correctas, y se necesita la experiencia de expertos para identificar la mejor.
- Enfoque: Se requiere análisis y la consulta de expertos para tomar decisiones. Las prácticas recomendadas pueden variar, pero se pueden identificar "buenas prácticas".
- Estrategia: Sentir-Analizar-Responder. Se recopilan datos, se analiza la situación y se elige la mejor práctica basada en el análisis experto.

3. Dominios Complejo

- Características: En situaciones complejas, la relación causa-efecto solo puede comprenderse retrospectivamente. El comportamiento emergente y la no linealidad son características clave, y no se puede predecir con precisión cómo se desarrollarán las cosas.
- Enfoque: La experimentación es fundamental, y las soluciones emergen a través de pruebas y adaptaciones. Es importante fomentar la creatividad y la innovación.
- Estrategia: Probar-Sentir-Responder. Se realizan experimentos seguros, se observa el resultado y se adapta en consecuencia. El objetivo es encontrar patrones y gestionar lo que surge.

4. Dominios Caótico

- Características: En el caos, no hay una relación discernible entre causa y efecto. La situación es altamente inestable y puede cambiar rápidamente.
- Enfoque: Se debe actuar rápidamente para estabilizar la situación, luego se puede pasar a otro dominio (como complejo o complicado) para gestionar el problema de manera más efectiva.

- Estrategia: Actuar-Sentir-Responder. Se toma acción inmediata para imponer orden, se observa el efecto de esa acción, y luego se responde según sea necesario.



Agilidad

No es una metodología, porque no es algo que pretenda indicarnos qué hacer paso a paso. NO es ir más rápido, aunque entregar valor temprano sí es una de las promesas ágiles. NO es multitasking, aunque implique gestionar un contexto complejo, incierto y cambiante. Es una forma de hacer, que nos invita a experimentar y aprender en pasos pequeños. Y una forma de ser. En la que nos ayudamos, nos cuidamos y mejoramos nuestro entorno. Es un mindset, que tienen quienes se enfocan en dar valor, trabajando colaborativamente.

Scrum

Es un marco de trabajo ágil utilizado en la gestión y desarrollo de proyectos, especialmente en el ámbito del software. Se centra en la entrega incremental de productos mediante ciclos cortos y repetitivos llamados sprints. Su principal objetivo es mejorar la flexibilidad, la colaboración y la calidad del producto final.

Eventos de Scrum:

- Sprint: Ciclo de trabajo corto (generalmente de 2 a 4 semanas) en el que el equipo desarrolla un incremento del producto.
- Sprint Planning: Reunión al inicio del sprint donde el equipo planifica qué trabajo se realizará durante ese sprint.
- Daily Scrum: Reunión diaria de 15 minutos donde el equipo sincroniza sus actividades y ajusta sus planes si es necesario.
- Sprint Review: Al final del sprint, el equipo muestra lo que ha logrado y recibe retroalimentación.
- Sprint Retrospective: Reunión al final del sprint para reflexionar sobre el proceso y mejorar en el próximo sprint.

Extreme Programming (XP)

La programación extrema es un método de desarrollo de software dividido en sprints de trabajo. Los marcos ágiles siguen un proceso iterativo, en el que se completa y revisa el marco al final de cada sprint, refinándolo para adaptarlo a los requisitos cambiantes y alcanzar la eficiencia máxima. Al igual que otros métodos ágiles, el diseño de la programación extrema permite a los desarrolladores responder a las solicitudes de los clientes, adaptarse y realizar cambios en tiempo

real. Sin embargo, la programación extrema es mucho más disciplinada; realiza revisiones de código frecuentes y pruebas unitarias para realizar cambios rápidamente. Además es muy creativa y colaborativa, ya que promueve el trabajo en equipo durante todas las etapas de desarrollo.

Claves:

- I. Programación en Parejas: Dos desarrolladores trabajan juntos en el mismo código para mejorar la calidad y reducir errores.
- II. Pruebas Unitarias: Se crean pruebas automáticas antes de escribir el código para asegurar que cada parte funcione correctamente.
- III. Integración Continua: El código se integra y se prueba varias veces al día para detectar y corregir problemas rápidamente.
- IV. Refactorización Continua: El código se mejora constantemente para mantenerlo limpio y flexible.
- V. Desarrollo Incremental: El software se entrega en ciclos cortos con versiones funcionales.
- VI. Propiedad Colectiva: Todos los desarrolladores pueden modificar cualquier parte del código en cualquier momento.
- VII. Cliente Involucrado: Un representante del cliente trabaja con el equipo para proporcionar retroalimentación continua.
- VIII. Semana de 40 Horas: Se evita el exceso de horas extra para mantener al equipo productivo.

Implementacion:

1. Configura el entorno para apoyar la programación en parejas y la integración continua.
2. Involucra al cliente en el proceso, asegurando su participación continua.
3. Desarrolla y usa pruebas unitarias desde el inicio.
4. Refactoriza el código constantemente para mantenerlo limpio.
5. Organiza el trabajo en ciclos cortos para entregar incrementos funcionales.
6. Integra el código continuamente para detectar errores lo antes posible.

Kanban

Es una metodología ágil que se utiliza para gestionar el trabajo de manera eficiente, visualizando el flujo de tareas y optimizando los procesos. Se enfoca en la mejora continua y en la flexibilidad, permitiendo a los equipos adaptarse a los cambios y gestionar el trabajo en curso de manera efectiva.

1. Visualización del Trabajo: Representa las tareas en un tablero Kanban para que todos puedan ver el estado del trabajo fácilmente.
2. Límite de Trabajo en Proceso (WIP): Restringe la cantidad de tareas que pueden estar en progreso al mismo tiempo para evitar la sobrecarga.
3. Gestión del Flujo: Optimiza el movimiento de las tareas a través del proceso, identificando y eliminando cuellos de botella.
4. Mejora Continua: Revisa y ajusta regularmente el proceso para aumentar la eficiencia y la calidad del trabajo.
5. Claridad de Políticas: Define y comunica claramente las reglas y procesos para mover tareas entre etapas.

Corazón de la Agilidad

- Colaborar: Fomenta la comunicación abierta y la cooperación entre el equipo y las partes interesadas.
- Entregar: Realiza entregas frecuentes de valor para recibir retroalimentación temprana.
- Reflexionar: Evalúa regularmente el trabajo realizado para identificar áreas de mejora.
- Mejorar: Aplica mejoras continuas basadas en la reflexión para incrementar la eficiencia y calidad.



Modern Agile

es un enfoque contemporáneo para el desarrollo de software y la gestión de proyectos que se basa en principios ágiles, pero con un enfoque más amplio y actualizado. Fue desarrollado por Joshua Kerievsky, y se centra en cuatro principios fundamentales que buscan maximizar el valor, la eficiencia y la satisfacción del cliente.

- Haz que las personas sean increíbles: Empodera y motiva a los miembros del equipo para que sean efectivos y felices.
- Entrega valor de manera continua: Realiza entregas frecuentes de valor a los clientes para obtener feedback constante.
- Haz que la seguridad sea un requisito: Crea un entorno seguro y confiable para la colaboración y la innovación.
- Experimenta y aprende rápidamente: Fomenta la experimentación y el aprendizaje rápido para mejorar continuamente.

SCRUM Ceremonias

Las ceremonias de Scrum son eventos estructurados que ayudan a los equipos a trabajar de manera eficiente y colaborativa, siguiendo los principios de Scrum. Aquí te explico cada una de las ceremonias, incluyendo lo que se hace y quiénes participan:

1. Sprint Planning (Planificación del Sprint)

- Objetivo: Definir qué se va a hacer en el próximo Sprint y cómo se va a lograr.
- Participantes: Todo el equipo Scrum, que incluye al Product Owner, el Scrum Master y el equipo de desarrollo.
- Actividades:

- Revisión del Product Backlog: El Product Owner presenta los ítems del Product Backlog que están priorizados.
- Selección de Ítems del Sprint Backlog: El equipo de desarrollo selecciona los ítems que se comprometerán a completar durante el Sprint.
- Definición del Objetivo del Sprint: El equipo define un objetivo claro para el Sprint, que guiará el trabajo durante el mismo.
- Planificación del Trabajo: El equipo desglosa los ítems seleccionados en tareas más pequeñas y crea un plan para completarlas.

2. Daily Scrum (Reunión Diaria)

- Objetivo: Coordinar el trabajo diario del equipo y ajustar el plan si es necesario.
- Participantes: El equipo de desarrollo principalmente, aunque el Scrum Master y el Product Owner pueden participar como observadores.
- Actividades:
 - Cada miembro responde a tres preguntas clave:
 - ¿Qué hice ayer?
 - ¿Qué voy a hacer hoy?
 - ¿Hay algún impedimento que me impida avanzar?
 - Duración: Es una reunión breve, usualmente de 15 minutos.
 - Formato: La reunión es de pie (stand-up) para fomentar que sea corta.

3. Sprint Review (Revisión del Sprint)

- Objetivo: Inspeccionar el Incremento de producto desarrollado durante el Sprint y adaptar el Product Backlog si es necesario.
- Participantes: Todo el equipo Scrum, stakeholders (interesados) y otras partes relevantes.
- Actividades:
 - Demostración del Incremento: El equipo de desarrollo muestra el trabajo completado durante el Sprint.
 - Comentarios y Feedback: Los stakeholders y el Product Owner dan retroalimentación sobre el Incremento.
 - Actualización del Product Backlog: Basado en el feedback, el Product Owner puede ajustar el Product Backlog.

4. Sprint Retrospective (Retrospectiva del Sprint)

- Objetivo: Reflexionar sobre el Sprint anterior y mejorar continuamente el proceso.
- Participantes: Todo el equipo Scrum.
- Actividades:
 - Revisión de lo que salió bien, lo que no salió bien y lo que se podría mejorar.
 - Identificación de acciones de mejora: El equipo propone y acuerda acciones concretas para mejorar en el próximo Sprint.
 - Aplicación de Mejoras: Las mejoras identificadas se implementan en el siguiente Sprint.

Resumen de Roles:

- Product Owner: Responsable de maximizar el valor del producto y del trabajo del equipo de desarrollo.

- Scrum Master: Facilita las ceremonias, asegura que Scrum se entienda y se aplique correctamente, y elimina impedimentos.
- Equipo de Desarrollo: Desarrolladores que realizan el trabajo técnico y entregan un Incremento de producto funcional al final de cada Sprint.

Product Discovery

El Product Discovery (o Descubrimiento de Producto) tiene como objetivo acercarse a los usuarios o destinatarios con el fin de identificar riesgos y oportunidades durante el desarrollo de un producto o servicio. En otras palabras, es un método que se utiliza para generar ideas innovadoras, ya sea para la creación de un producto o servicio como para la resolución o mejoras de problemas o desafíos existentes. La gran diferencia con otras metodologías es que se centra en las necesidades reales de los usuarios y se los involucra en la búsqueda de la innovación.

Un producto nunca está “listo”, está en constante evolución, siempre apuntando a entregar más valor a los clientes.

Product Discovery Mindset

1) Colaboración y aprendizaje en el Equipo de Producto

- Todo el Equipo de Producto conoce y aprende del Product Discovery, participando en las decisiones sobre la funcionalidad a crear
- Evitando guerras de opiniones y corazonadas
- Desarrollando y manteniendo un entendimiento compartido
- Con artefactos de visualización

2) User Centered Design

- El Equipo de Producto trabaja en comunicación directa con usuarios y clientes
- No sólo validando problemas y soluciones sino además co-creando junto a ellos
- Diseñar experiencias

3) Humildad intelectual

- Tolerancia a la incertidumbre: no hay certezas sino hipótesis a validar
- Experimentos: tolerancia al error y a cambiar de parecer
- Construir un ambiente seguro e inclusivo de respeto a las opiniones de todo el equipo

UX Personas

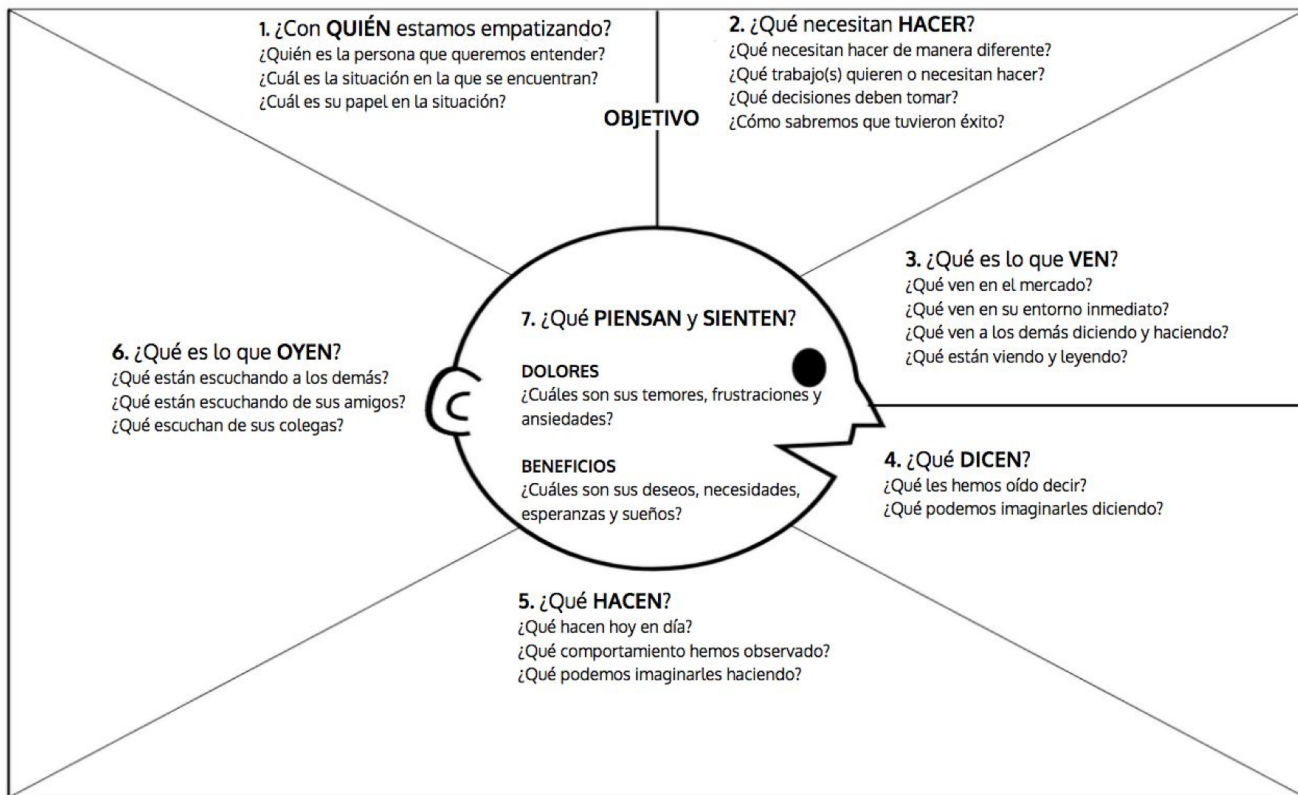
- Hacer foco en segmentos específicos.
- Técnica para comprender las diferentes necesidades de los principales tipos de clientes de un producto.

- También sirve para generar conversaciones valiosas al momento de definir y priorizar las oportunidades.

 Datos <nombre> <edad> <Profesión> <... >	Problemas
Comportamiento	Necesidades y objetivos

Mapa de Empatía

- Tablero para conocer quién es tu cliente
- Qué problemas y necesidades tiene
- Y qué propuesta de valor le ofreceremos



Historias de Usuario

Descripción de la funcionalidad del sistema, desde el punto de vista de un usuario:

Como <persona> quiero <función> para <objetivo>

Se enfocan en el valor provisto, dando pie a discutir el requisito No son el requisito en sí .

Calidad Sintáctica	Bien formadas	Atómicas
	Mínimas	
Calidad Semántica	Conceptualmente Acertadas	Orientadas al Problema
	Sin Ambigüedades	Sin Conflictos
Calidad Pragmática	Usan Oraciones Completas	Estimables
	Únicas	Uniformes
	Independientes	Completas

Épicas: Historias de usuario que se pueden descomponer en más historias de usuario. Deseamos subdividir las épicas hasta que llegamos a tareas de tamaño razonable.

Criterios INVEST

I	ndependent	Separable de otras tareas
N	egotiable	No es innecesariamente rígido
V	aluable	Aporta valor al cliente/usuario
E	stimable	Posible aproximar su costo
S	mall	Desarrollable en una iteración
T	estable	Verificable (al menos en principio)

3 Cs (Card, Conversation, Confirmation)

Card: Describe la intención del usuario.

Conversation: Los interesados se comunican para refinar las historias, descubriendo y documentando requisitos.

Confirmation: Criterios de aceptación.

Criterios de Aceptación

Condiciones específicas que deben cumplirse para aceptar trabajo realizado:

- Listas de condiciones
- Escenarios

Dado que <contexto> cuando suceda <evento> entonces <consecuencia>

- Deben ser:
 - Claros
 - Concisos
 - Verificables
 - Independientes
 - Orientados al Problema

Behaviour Driven Development

Es un proceso de software ágil que busca la colaboración y entendimiento entre desarrolladores, gestores de proyecto y equipo de negocio. Es decir, es el camino para tomar antes de la fase de testing de un proyecto.

BDD describe las pruebas en un lenguaje natural que entienden todos los equipos de un proyecto, y no únicamente los programadores.

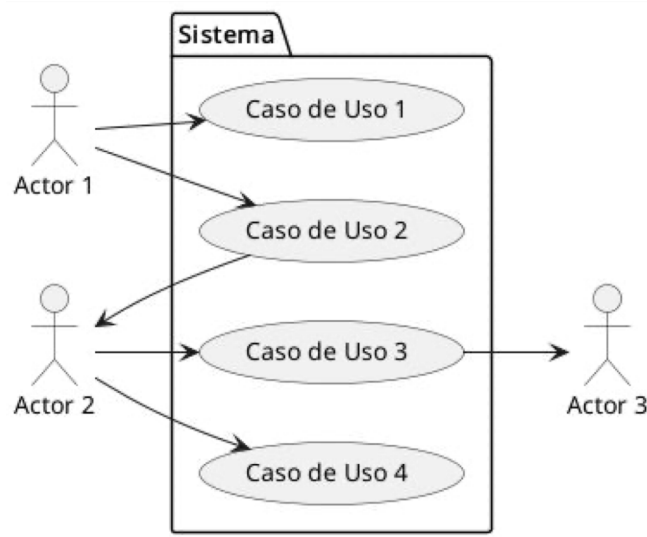
Casos de Uso

Descripción de:

- Una serie de acciones.
- Realizadas por un sistema.
- Que generan un resultado observable de valor para un actor en particular.

Compuestos de un escenario principal y un conjunto de escenarios alternativos.

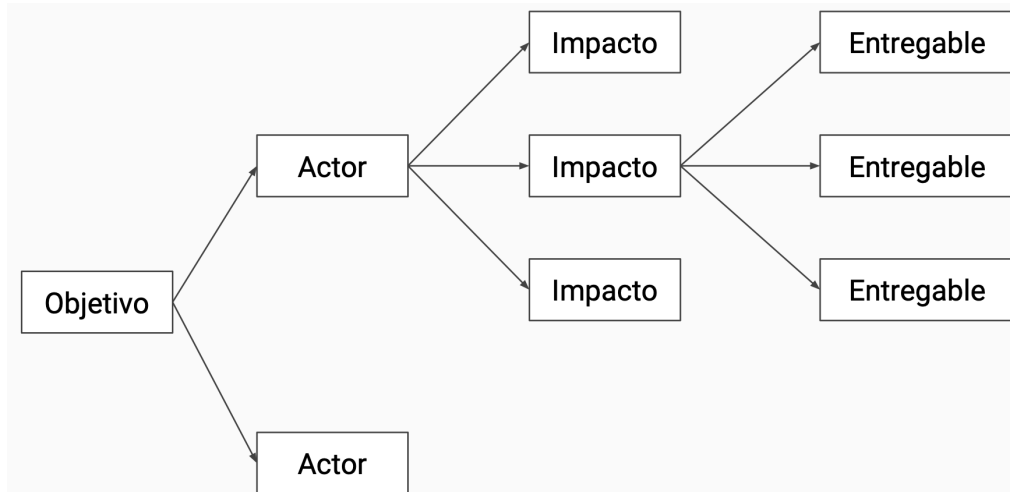
Diagrama de Casos de Uso



User Story Mapping



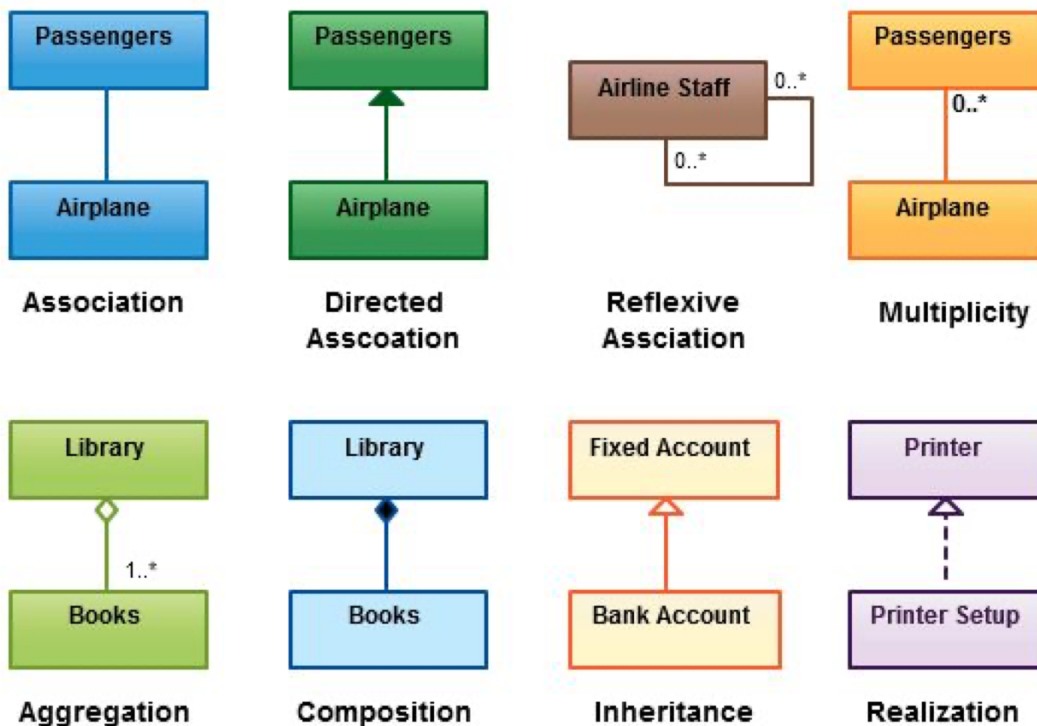
Impact Mapping



Diagramas UML

El lenguaje unificado de modelado o UML es una forma de representar visualmente la arquitectura y diseño de sistemas de software complejos. Facilita la comprensión de ideas y sistemas complejos mediante la visualización.

Relaciones



Relaciones en los diagramas de clase UML

Diagramas estructurales

Diagrama de clases

Muestra las clases, sus atributos, métodos y visibilidad (pública, privada, protegida). Ilustra las conexiones entre clases, como herencia, composición, asociación y dependencia. Ayuda a planificar el diseño de software y comunicar cómo se relacionan y funcionan las diferentes partes del sistema.

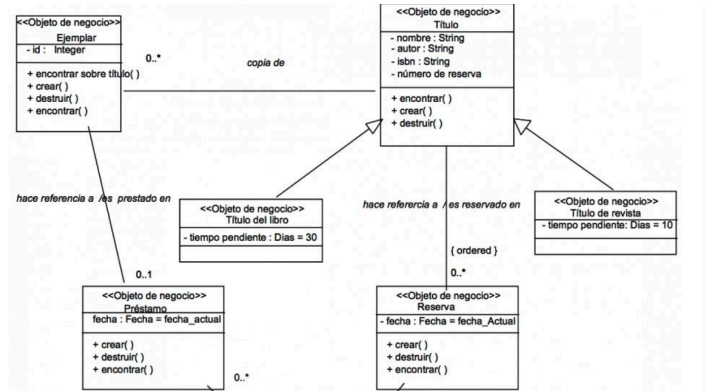


Diagrama de paquetes

Se utiliza para organizar y agrupar elementos de un sistema en diferentes módulos o paquetes. Su propósito es mostrar cómo se dividen las partes del sistema en unidades lógicas o funcionales, y cómo esos paquetes se relacionan entre sí. Un paquete puede contener clases, interfaces, otros paquetes, entre otros elementos, y los diagramas de paquetes se usan para visualizar las dependencias y relaciones entre estos grupos de elementos.

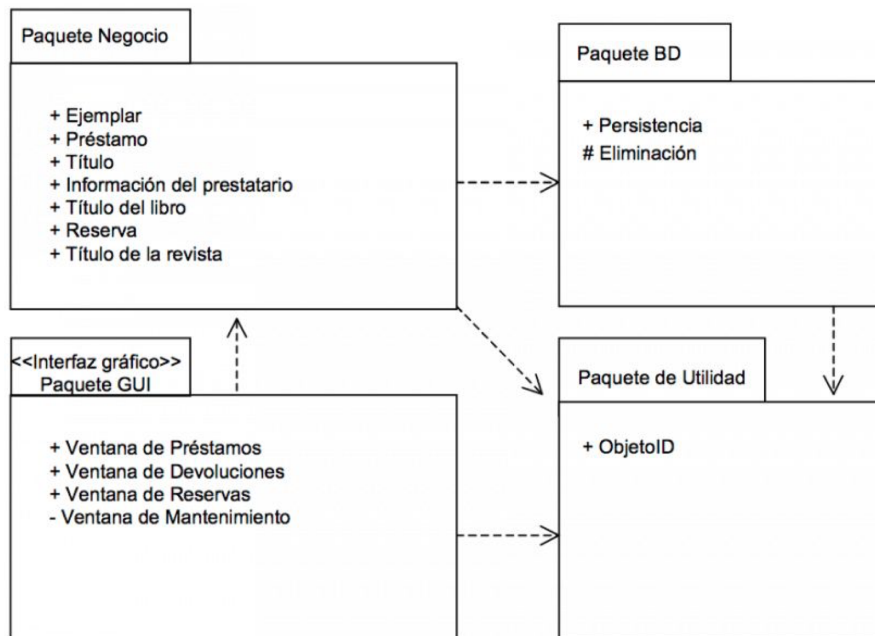


Diagrama de objetos

Muestra una instantánea del estado de los objetos en un sistema en un momento dado. A diferencia del diagrama de clases, que se enfoca en la estructura y definición general de las clases, el diagrama de objetos muestra instancias concretas de esas clases y sus valores actuales.

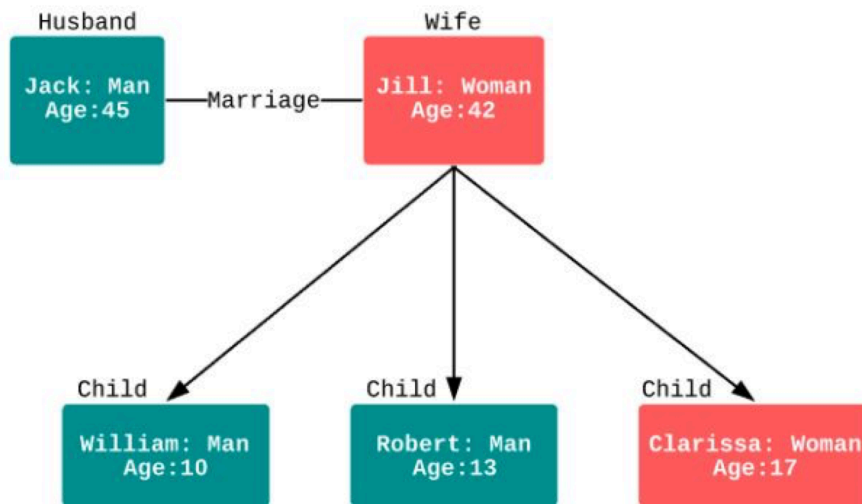


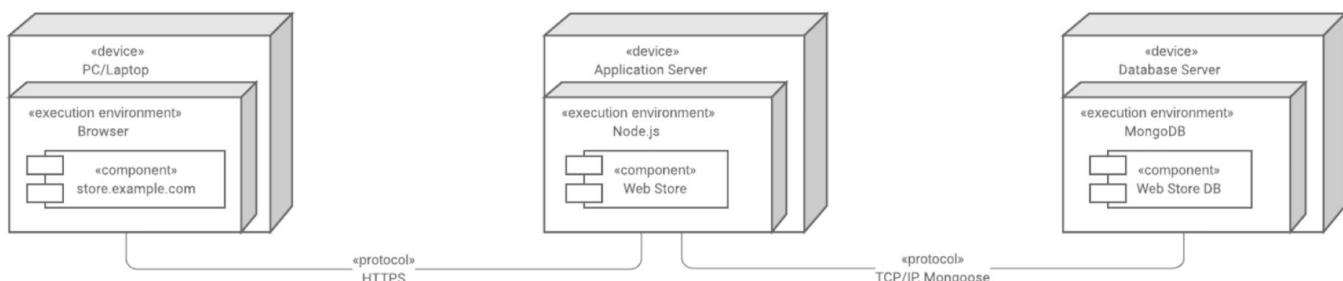
Diagrama de componentes

Muestra la estructura física de un sistema de software al ilustrar los diferentes componentes del sistema y cómo interactúan entre sí. Un componente es una parte modular e independiente del sistema, que puede ser un archivo, una librería, un módulo, o un servicio. Estos diagramas representan cómo se agrupan y conectan los distintos componentes en un sistema complejo.



Diagrama de despliegue

Muestra cómo se distribuyen los componentes de un sistema en su infraestructura física, es decir, en servidores, dispositivos de red, u otras plataformas de hardware. Este tipo de diagrama representa la topología física del sistema, mostrando los nodos (por ejemplo, servidores o dispositivos), y cómo se conectan e interactúan a través de redes o enlaces.



Diagramas de comportamiento

Diagrama de actividades

representa el flujo de trabajo o el comportamiento dinámico de un proceso. Muestra las secuencias de acciones y las decisiones que ocurren en un sistema o en una parte del sistema. Es similar a un

diagrama de flujo, pero se usa para describir procesos dentro del contexto del modelado orientado a objetos.

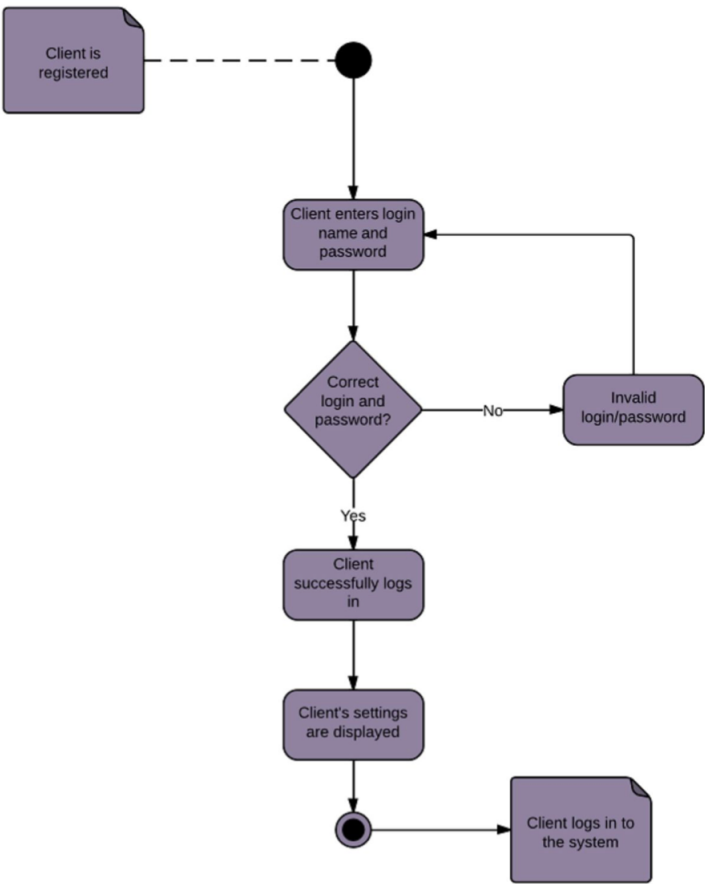


Diagrama de secuencia

Tipo de diagrama de interacción que ilustra cómo los objetos de un sistema intercambian mensajes entre sí en una secuencia cronológica. Representa la dinámica del sistema, mostrando el flujo de eventos o interacciones que ocurren en un escenario específico, como un caso de uso.

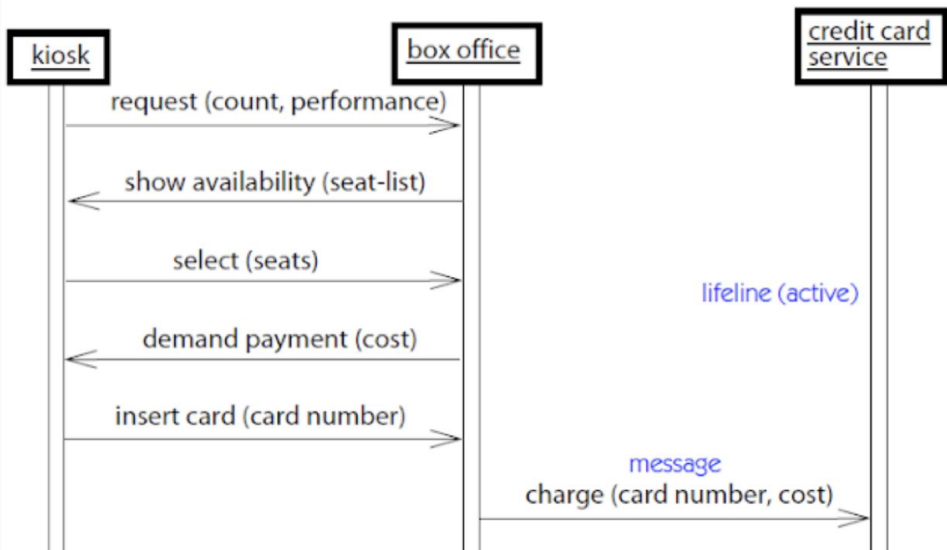


Diagrama de casos de uso

Representa las interacciones entre los usuarios (actores) y el sistema. Muestra qué acciones o funciones realiza el sistema desde la perspectiva del usuario, pero sin entrar en detalles sobre cómo se implementan internamente.

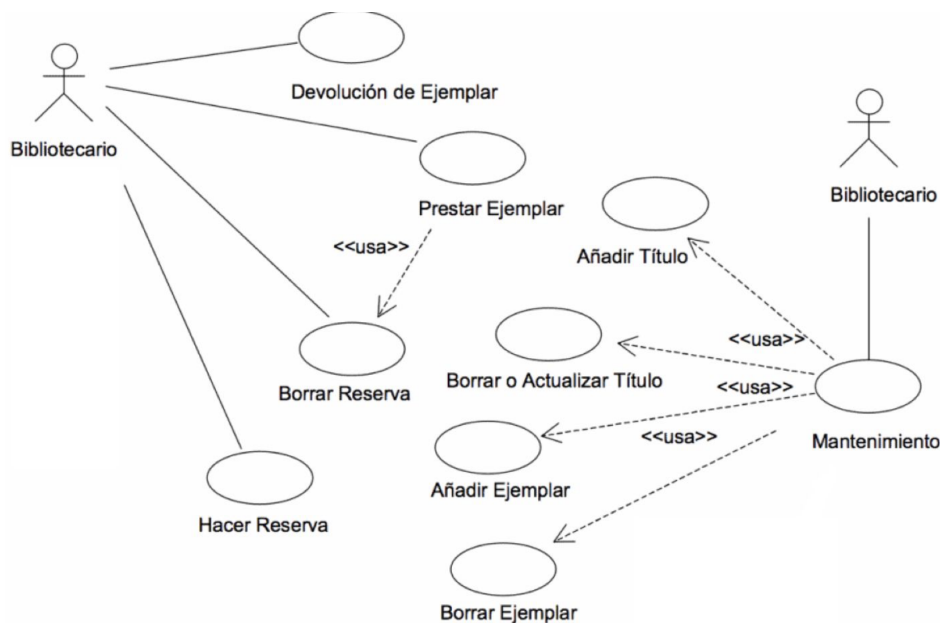
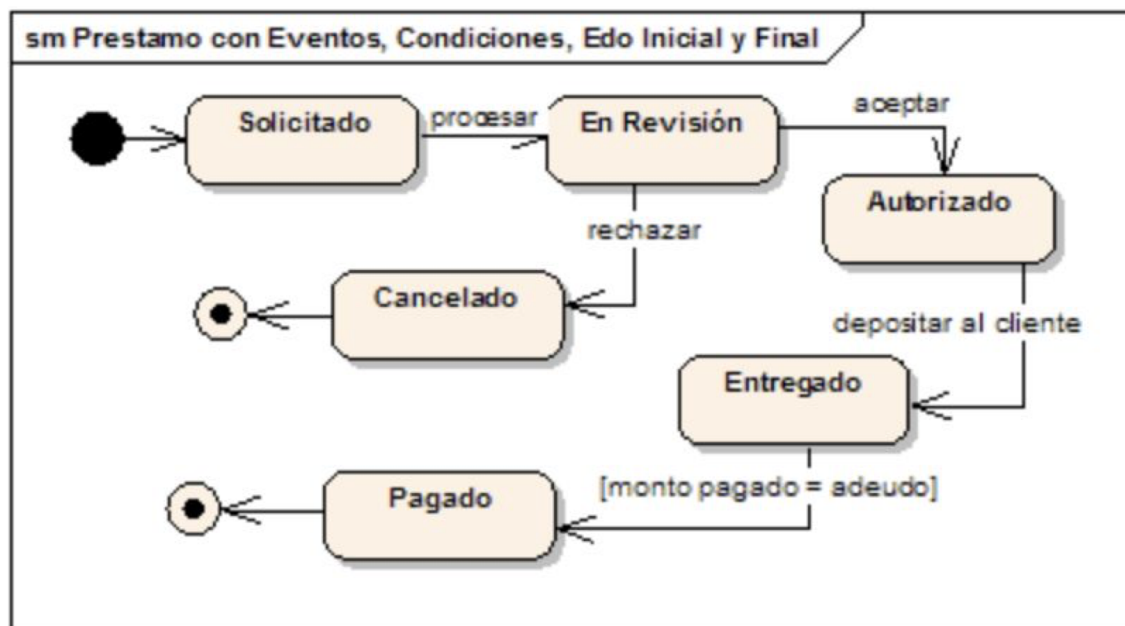


Diagrama de máquina de estados

Muestra los estados por los que puede pasar un objeto a lo largo de su vida y las transiciones entre estos estados en respuesta a eventos. Representa el comportamiento dinámico de un objeto o sistema, enfocándose en cómo cambia su estado en respuesta a eventos internos o externos.



Arquitectura de Software

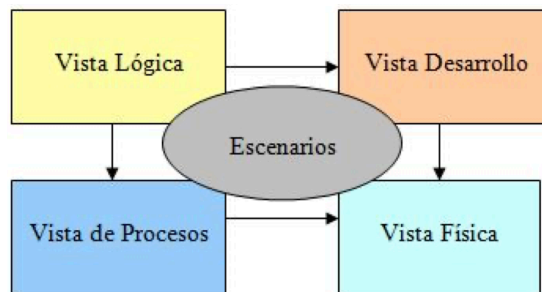
Conceptos o propiedades fundamentales de un sistema en su entorno encarnado en sus elementos, relaciones y en los principios de su diseño y evolución. La arquitectura de software representa la estructura o las estructuras del sistema, que consta de componentes de software, las propiedades visibles externamente y las relaciones entre ellas.

La arquitectura de software son aquellas decisiones que son importantes y difíciles de cambiar.

Martin Fowler

En realidad existen muchos interesados o stakeholders de un documento de arquitectura, e incluso estos pueden tener distintos intereses y conocimientos. ¿cómo escribir un único documento o diagrama que pueda explicar la arquitectura propuesta para resolver cierto problema?

Modelo de Vistas 4+1 - Kruchten



VISTA LOGICA

La vista lógica se enfoca en los requisitos funcionales del sistema, mostrando los servicios que debe ofrecer a los usuarios. Utiliza abstracción, encapsulamiento y herencia para descomponer el sistema, facilitando el análisis funcional y ayudando a identificar mecanismos y elementos de diseño comunes.

VISTA DE PROCESOS

La vista de procesos se centra en los requisitos no funcionales como rendimiento y disponibilidad. Aborda temas de concurrencia, distribución, integridad del sistema y tolerancia a fallas. Un proceso agrupa tareas en una unidad ejecutable, permitiendo la gestión táctica (inicio, recuperación, reconfiguración y detención). Además, los procesos pueden replicarse para distribuir la carga o mejorar la disponibilidad.

VISTA DE DESARROLLO (o de COMPONENTES)

La vista de desarrollo se centra en la organización real del software en el entorno de desarrollo. Muestra cómo el software se divide en módulos (como bibliotecas o subsistemas) que pueden ser manejados por pequeños equipos. Considera requisitos internos como facilidad de desarrollo, administración del software, reutilización, y las restricciones de las herramientas o lenguajes de programación utilizados.

VISTA FISICA (o de DESPLIEGUE)

La vista física se enfoca en los requisitos no funcionales como disponibilidad, confiabilidad, rendimiento y escalabilidad. Muestra cómo el software se distribuye sobre una red de nodos y cómo los elementos del sistema se asignan a estos nodos. Permite configuraciones flexibles para desarrollo, pruebas y despliegue, con un impacto mínimo en el código fuente.

VISTA DE ESCENARIOS

Las cuatro vistas trabajan juntas utilizando un conjunto de escenarios relevantes, que abstraen los requisitos importantes. Estos escenarios sirven para guiar el diseño arquitectónico y para validar e ilustrar el diseño una vez completado, además de ser el punto de partida para las pruebas de un prototipo.

Decisiones de Arquitectura

Son decisiones de diseño que abordan requisitos significativos desde el punto de vista arquitectónico; se perciben como difíciles de hacer y/o costosos de cambiar.

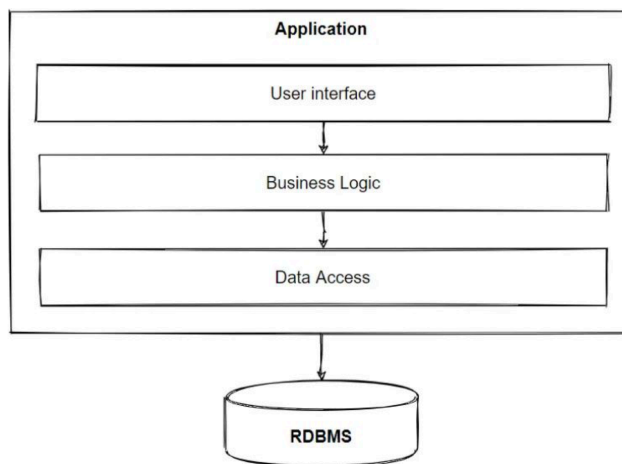
Estas decisiones son importantes de guardarlas y conocerlas. Conocer el motivo por el cual se tomó dicha decisión en la arquitectura o diseño.

Software Architecture Patterns

La arquitectura de software es el diseño fundamental de un sistema, que define cómo se dividen sus componentes y cómo se comunican entre sí. Implica tomar decisiones de diseño significativas y es evaluada por el costo de cambio que implica modificar el sistema. En esencia, moldea el sistema en su totalidad.

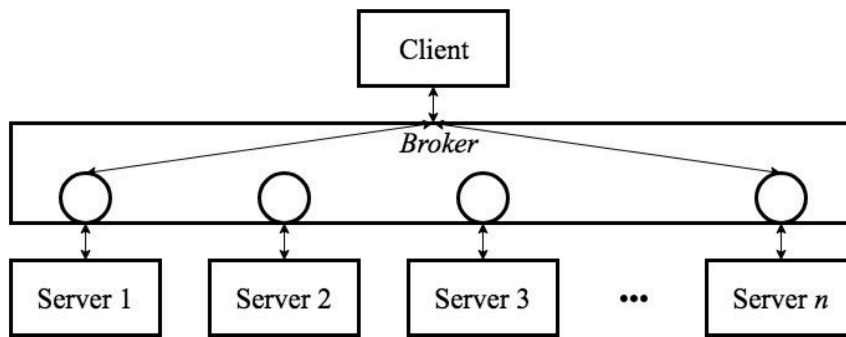
Layers

El patrón de diseño de capas (layers) organiza un sistema en niveles jerárquicos o capas, donde cada capa tiene una responsabilidad específica. Las capas están estructuradas de manera que cada una solo interactúa con la capa inmediatamente inferior y superior. Esto permite modularidad, mantenimiento más sencillo y una separación clara de las preocupaciones. Generalmente, las capas incluyen la capa de presentación, la capa de lógica de negocio y la capa de acceso a datos.



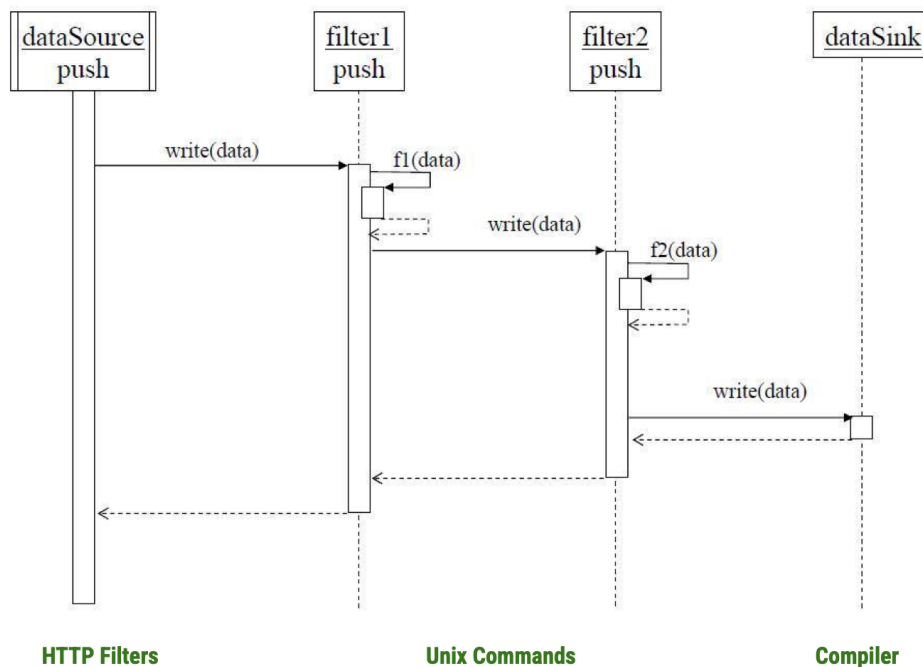
Broker

El patrón de diseño de broker se utiliza para desacoplar los componentes de un sistema distribuidos o en red. Un bróker actúa como intermediario entre clientes y servicios, facilitando la comunicación y coordinación entre ellos. Los clientes envían solicitudes al broker, que luego las redirige al servicio apropiado y devuelve las respuestas al cliente. Este patrón permite una flexibilidad en la arquitectura, ya que los componentes pueden comunicarse sin necesidad de conocer detalles específicos sobre los otros, y facilita la gestión de la comunicación en sistemas distribuidos.



Pipe & Filters

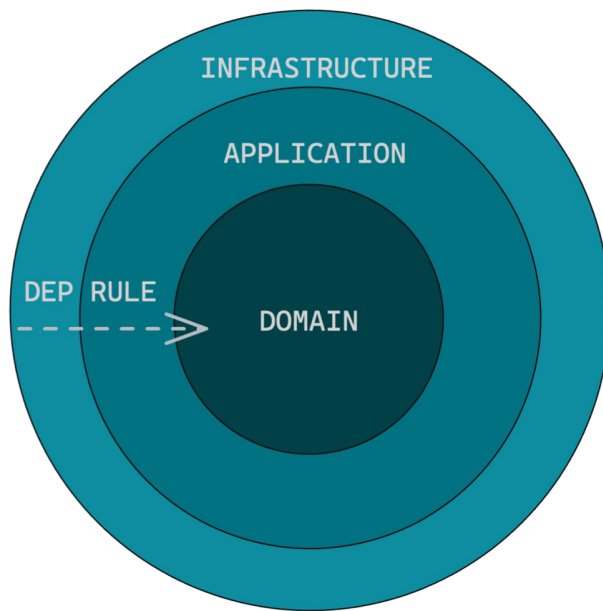
El patrón de diseño de pipe and filter organiza un sistema como una serie de filtros conectados por tuberías (pipes). Cada filtro es un componente que procesa datos de entrada y produce datos de salida, mientras que las tuberías transportan los datos de un filtro al siguiente. Este patrón permite la composición de procesos complejos a partir de componentes simples y reutilizables, facilita el encadenamiento de procesos y mejora la modularidad y mantenibilidad del sistema. Es ideal para sistemas que requieren un procesamiento secuencial o en flujo de datos.



Hexagonal Architecture

La Arquitectura Hexagonal, también conocida como arquitectura de puertos y adaptadores, organiza el sistema en un núcleo central que contiene la lógica de negocio, rodeado de adaptadores que permiten la interacción con el mundo exterior. El núcleo no depende de ninguna tecnología externa; en cambio, las interfaces para entrada y salida están definidas por puertos que los adaptadores implementan.

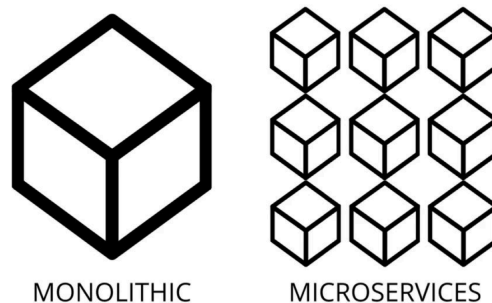
Este enfoque promueve desacoplamiento y flexibilidad, ya que el núcleo de la aplicación puede ser modificado sin afectar los adaptadores externos, y viceversa. Facilita el testeo y la sustitución de componentes externos, como bases de datos o interfaces de usuario, sin alterar la lógica central del sistema.



Microservices

El patrón de microservicios divide una aplicación en servicios pequeños e independientes, cada uno de los cuales realiza una función específica y se comunica con otros servicios a través de interfaces bien definidas (como APIs). Cada microservicio puede ser desarrollado, desplegado y escalado de manera independiente, lo que permite una mayor flexibilidad y resiliencia en el sistema.

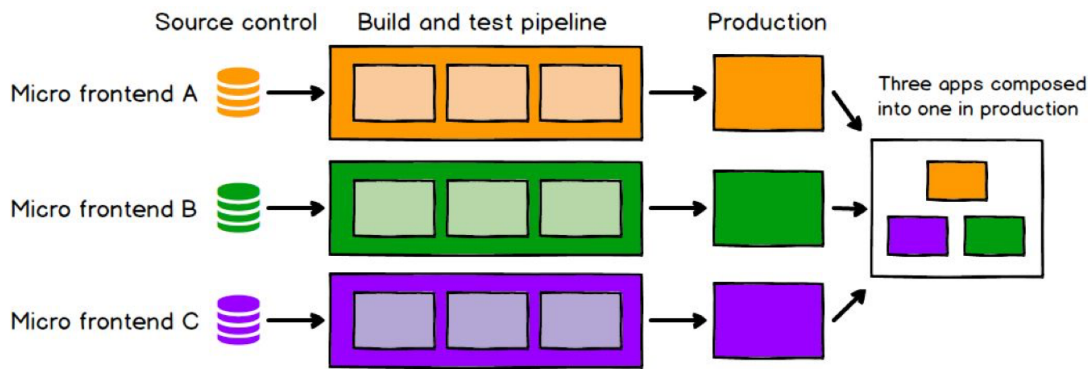
Este enfoque facilita la escalabilidad y mantenimiento, ya que los cambios en un servicio no afectan directamente a los demás. También permite el uso de diferentes tecnologías y lenguajes para distintos servicios, adaptándose a necesidades específicas. Sin embargo, la gestión de la comunicación entre servicios y la coordinación pueden volverse complejas a medida que el número de microservicios aumenta.



Micro Frontends

El patrón de Micro Frontends extiende el concepto de microservicios al desarrollo de interfaces de usuario. En lugar de una sola aplicación frontend monolítica, se divide la interfaz en componentes o módulos independientes, cada uno desarrollado, desplegado y gestionado por equipos distintos.

Cada micro frontend puede manejar una parte específica de la interfaz, como una sección de un dashboard o una funcionalidad específica, y se integra con otros micro frontends a través de interfaces bien definidas (como APIs o eventos). Esto permite una mayor escalabilidad y flexibilidad, ya que diferentes partes de la interfaz pueden ser actualizadas o escaladas de manera independiente.



REST

REST (Representational State Transfer) es un estilo de arquitectura para diseñar servicios web que aprovechan los principios y protocolos de la web. Fue propuesto por Roy Fielding en su tesis doctoral en el año 2000. REST se basa en una serie de principios y restricciones que facilitan la interacción entre sistemas, aprovechando HTTP como protocolo de comunicación.

Stateless

Implica que el servidor no mantiene ninguna información o contexto sobre la interacción pasada del cliente entre solicitudes. Cada petición (request) que un cliente envía al servidor debe ser tratada de forma independiente, como si fuera la primera vez que se realiza.

Dado que no se almacena información de sesión, cada request se ejecuta de forma autónoma y no depende de solicitudes anteriores. Para que cada solicitud pueda procesarse correctamente, el cliente debe incluir toda la información necesaria dentro de la misma solicitud.

Cacheable

Es la capacidad de las respuestas del servidor para ser almacenadas temporalmente en caché, lo que optimiza el rendimiento general de la aplicación. Al permitir que las respuestas se almacenen en caché, se reduce la necesidad de realizar solicitudes repetidas al servidor, lo que disminuye el uso de ancho de banda y reduce la carga en los servidores, mejorando así su eficiencia.

Además reduce la latencia al permitir que los clientes obtengan rápidamente respuestas almacenadas localmente. Esto también puede ocultar fallos temporales de red.

Compresión

Se utiliza para reducir el tamaño de las respuestas que el servidor envía al cliente, lo que optimiza el uso del ancho de banda y acelera las transferencias de datos. Las APIs pueden devolver los datos en varios formatos, como JSON, XML, HTML, o texto plano, y estos pueden ser comprimidos antes de ser enviados al cliente.

El cliente puede informar al servidor qué tipos de compresión soporta mediante el encabezado Accept-Encoding. El servidor, por su parte, cuando responde, usa el encabezado Content-Encoding para especificar qué tipo de compresión utilizó.

URIS

Los URIs permiten acceder a estos recursos a través de una estructura coherente y estandarizada. En las APIs REST, los URIs se usan para representar los recursos de manera clara y organizada. Es una cadena de caracteres que identifica de manera única un recurso en una API o en la web.

Por convención, los URIs usan sustantivos en plural para referirse a los recursos, evitando el uso de verbos. Esto ayuda a mantener una semántica clara y consistente. Además, los URIs suelen hacer distinción entre recursos principales y subordinados, organizando jerárquicamente los recursos de acuerdo con sus relaciones lógicas.

Verbos HTTP – Requests

Los verbos HTTP se utilizan para definir las acciones que se realizan sobre los recursos en una API. Cada uno tiene un propósito específico:

- GET solicita la representación de un recurso específico, sin modificarlo.
- POST se usa para enviar datos al servidor y crear un nuevo recurso.
- DELETE elimina un recurso específico.
- PUT reemplaza completamente un recurso existente con los datos enviados en la solicitud.
- PATCH realiza modificaciones parciales sobre un recurso, en lugar de reemplazarlo completamente como lo hace PUT.
- OPTIONS describe las opciones de comunicación disponibles para el recurso, informando qué métodos HTTP son soportados.

HTTP Status Codes - Responses

- 1xx: Informational
- 2xx: Success
- 3xx: Redirection
- 4xx: Client Error
- 5xx: Server Error

principios de seguridad en el diseño REST

- Least Privilege asegura que cada usuario o servicio tenga los mínimos permisos necesarios para realizar sus acciones, limitando el acceso innecesario.
- Fail-Safe Defaults establece que, por defecto, el acceso a los recursos debe estar restringido, permitiendo acceso solo cuando se autoriza explícitamente.
- Complete Mediation significa que cada intento de acceso a un recurso debe ser validado y autorizado por el sistema, sin suposiciones de permisos previos.
- Keep it Simple promueve la simplicidad en el diseño de seguridad, minimizando la probabilidad de errores.
- El uso de HTTPS garantiza la encriptación de las comunicaciones, protegiendo los datos en tránsito.
- Password hashes (como PBKDF2, bcrypt, y scrypt) son utilizados para asegurar las contraseñas, en lugar de almacenarlas en texto claro.
- Nunca exponer información sensible en las URLs, como nombres de usuario, contraseñas, tokens de sesión o claves de API, ya que podrían quedar expuestos en logs del servidor.

- Timestamp en los requests puede ayudar a prevenir ataques de repetición, asegurando que las solicitudes no puedan ser reutilizadas en el tiempo.
- Validación de parámetros de entrada es fundamental para evitar ataques como la inyección de código malicioso o datos inválidos que comprometan la seguridad de la API.

Autenticación & Autorización

- Basic Auth: Es un método sencillo en el que el cliente envía las credenciales (usuario y contraseña) codificadas en base64 en el encabezado Authorization. Es fácil de implementar, pero inseguro si no se usa junto con HTTPS, ya que las credenciales se transmiten en cada solicitud.
- API Keys: Se utilizan como identificadores únicos para clientes o servicios que acceden a la API. El cliente envía la clave (key) en cada solicitud, generalmente en el encabezado o como parámetro de la URL. Las API Keys proporcionan una manera fácil de autenticar, pero no son tan seguras por sí solas, ya que pueden ser expuestas si no se manejan adecuadamente.
- Bearer Authentication: En este método, el cliente envía un token (generalmente generado por el servidor al inicio de la sesión) en el encabezado Authorization con el formato Bearer <token>. Este token es validado por el servidor en cada solicitud, y es una forma más segura de manejar autenticación.
- OAuth: Es un protocolo estándar para autorización. OAuth permite que aplicaciones de terceros accedan a los recursos de un usuario sin que necesiten conocer su contraseña. El cliente obtiene un token de acceso tras autenticarse, y este token es utilizado para realizar solicitudes autorizadas.
- JWT (JSON Web Tokens): Es un tipo de token utilizado comúnmente en la autenticación y autorización. Un JWT contiene información en forma de "claims" (reclamaciones) y está firmado digitalmente, lo que permite verificar su integridad. Los JWT pueden ser utilizados como Bearer tokens y son autocontenidos, por lo que no requieren almacenamiento en el servidor.

Refresh Tokens

Los access token deberían tener un tiempo limitado de vida, por eso aparecen los refresh token. Este es otro token que sirve para un solo uso y es utilizado para obtener un nuevo access token. Es una credencial que permite obtener nuevos tokens sin necesidad de usar las credenciales de usuario y password nuevamente.

Versionado

No hay una única forma definida, pero existen varias estrategias comunes para gestionar las diferentes versiones de una API. Estas son algunas de las más utilizadas:

Usando la URI: La versión de la API se incluye directamente en la URL. Esto es fácil de entender y visualizar para los usuarios, ya que la versión es parte del endpoint. Ej: <https://api.fi.uba.ar/v1>

Usando un Custom Header: El cliente especifica la versión que desea utilizar mediante un encabezado personalizado, como Accept-version. Esto mantiene la URL limpia, pero requiere que el cliente conozca y maneje adecuadamente el encabezado. Ej: `Accept-version: v1`

Usando el Header Accept: Aquí se utiliza el encabezado estándar Accept para especificar tanto el formato del contenido (por ejemplo, JSON) como la versión de la API. Es una técnica más sofisticada y flexible, pero puede ser menos intuitiva para los usuarios menos técnicos.

Ej: `Accept: application/vnd.example.v1+json`

Respuestas

En el diseño de respuestas para APIs REST, es fundamental seguir ciertos principios para asegurar que la comunicación sea clara y eficiente. Las respuestas deben tener un formato coherente y predecible, lo que facilita su manejo por parte de los clientes. Esto incluye usar la misma estructura para diferentes tipos de respuestas (éxitos y errores). Es importante que las respuestas sean concisas, enviando solo la información necesaria. Esto ayuda a optimizar el rendimiento y a reducir el uso de ancho de banda. Cada respuesta debe incluir un código de estado HTTP apropiado que refleje el resultado de la solicitud.

Paginado, filtros y ordenamientos

Son importantes para gestionar grandes conjuntos de datos de manera eficiente.

El paginado permite dividir los resultados en páginas más pequeñas, lo que mejora el rendimiento y la usabilidad. Los filtros y ordenamientos suelen implementarse como parámetros en el query string de la URL o en el cuerpo de la solicitud.

Logging, Health y Metrics

Son fundamentales para el monitoreo y la gestión de la API. Los logs permiten registrar actividades y errores, lo que ayuda a diagnosticar problemas. Las métricas proporcionan datos sobre el rendimiento y el uso de recursos, permitiendo detectar anomalías antes de que se conviertan en problemas graves.

Se pueden agregar endpoints específicos para estos propósitos, como `/health` y `/metrics`.

TDD

Test-Driven Development (TDD) es una técnica de desarrollo en la que primero se escriben las pruebas antes de implementar el código. El proceso sigue tres pasos principales:

1. Escribir una **prueba** que defina lo que el código debe hacer (y que inicialmente fallará).
2. Escribir el **código mínimo** necesario para que la prueba pase.
3. **Refactorizar** el código para mejorarlo sin romper la prueba.

Este ciclo se repite continuamente. TDD ayuda a asegurar que el código funcione correctamente desde el principio, mejora la calidad y facilita el mantenimiento del software.

Baby steps se refiere a avanzar en el desarrollo con pequeños cambios o mejoras incrementales. En lugar de hacer grandes cambios o escribir mucho código de una vez, se implementa y prueba cada funcionalidad en pasos muy pequeños.

Refactoring

Es el proceso de mejorar el código sin cambiar su comportamiento externo. Implica hacer que el código sea más claro, eficiente o mantenible, sin alterar lo que el software hace.

Para lograr una refactorización exitosa, es importante seguir estos dos pasos clave:

1. Usar **pruebas unitarias** para garantizar que el comportamiento externo del código no cambie. Estas pruebas actúan como una red de seguridad para asegurarse de que, aunque el código interno se modifique, la funcionalidad esperada sigue siendo la misma.

2. Aplicar las refactorizaciones propuestas, que implican mejorar la estructura del código sin modificar su funcionalidad.

El flujo de refactorización sigue estos pasos:

1. Asegurarse de que todas las pruebas pasen, para confirmar que el código actual funciona correctamente.
2. Identificar "code smells" o áreas problemáticas del código que necesitan mejoras.
3. Encontrar la refactorización adecuada para corregir esos problemas.
4. Aplicar la refactorización, mejorando la estructura del código sin cambiar su funcionalidad.

Un **code smell** es una señal de mal diseño en el código que puede complicar su mantenimiento o comprensión a largo plazo. Ejemplos comunes incluyen código duplicado, funciones demasiado largas, clases con demasiadas responsabilidades, nombres poco descriptivos, acoplamiento excesivo y código muerto. Aunque no son errores inmediatos, es recomendable corregirlos mediante refactorización para mejorar la calidad y mantenibilidad del software.